

Tic-Tac-Toe Analysis Project

1. Introduction

This project is based on the simple game Tic-Tac-Toe, but unlike the traditional version, it was designed to discourage perfect play by introducing limited information and random elements into the environment.

The players are two AIs(O and X) that utilize the minimax algorithm, each calculating the opponent’s optimal moves in an attempt to find the best possible strategy.

2. How It works

- **All Environment :**

Environment	Language	Analysis Libraries	Game Type	Algorithm
Visual Script Code	Python	pandas matplotlib.pyplot	TIC-TAC-TOE	minimax

- **Game Environment :**

Tic-Tac-Toe (Standard 3×3 grid)

- **AI Behavior :**

- **First move decision :**

The first player(O or X) is randomly selected at the start of each game.

```
def play_game():
    game = TicTacToe()
    letter = random.choice(["X", "O"])
```

- The AI uses the minimax algorithm to predict all possible moves and select the best outcome, but at each turn, there is a 10% chance it will make a completely random move.
 - “Depth” refers to how many moves ahead the AI will calculate. It is randomly set between 1 and 4 for each move, introducing variability in the AI’s decision-making process.

```
while game.empty_squares():
    if random.random() < 0.1:
        move = random.choice(game.available_moves())
    else:
        random_depth = random.randint(1, 4)
        move = minimax(game, letter, depth=random_depth)['position']
```

- **Simulation Execution :**

- Each set consisted of 50 games, and a total of 5 sets were conducted.

```
if __name__ == "__main__":
    n_sets = 5
    n_games_per_set = 50
```

- **Data Recording :**

- After each set, the number of wins for O,X and the number of draws were recorded and summarized. The results were also visualized using matplotlib.pyplot as bar graphs.

```
all_results = []

for i in range(n_sets):
    set_result = simulate_games(n_games_per_set)
    print(f"Set {i+1}: {set_result}")
    all_results.append(set_result)

df = pd.DataFrame(all_results, index=[f"Set {i+1}" for i in range(n_sets)])
print("\n=== Summary ===")
print(df)
```

```
df.plot(kind="bar", figsize=(10,6))
plt.title("Tic-Tac-Toe AI Simulation Results (5 Sets)")
plt.xlabel("Set")
plt.ylabel("Number of Wins / Draws")
plt.xticks(rotation=0)
plt.legend(title="Result")
plt.tight_layout()
plt.show()
```

3. Analysis

After running the program, the average results across 5 sets were : 24 wins for O, 22 wins for X, and 4 draws. Despite introducing randomness, there was no significant bias in win rates between the two AIs.

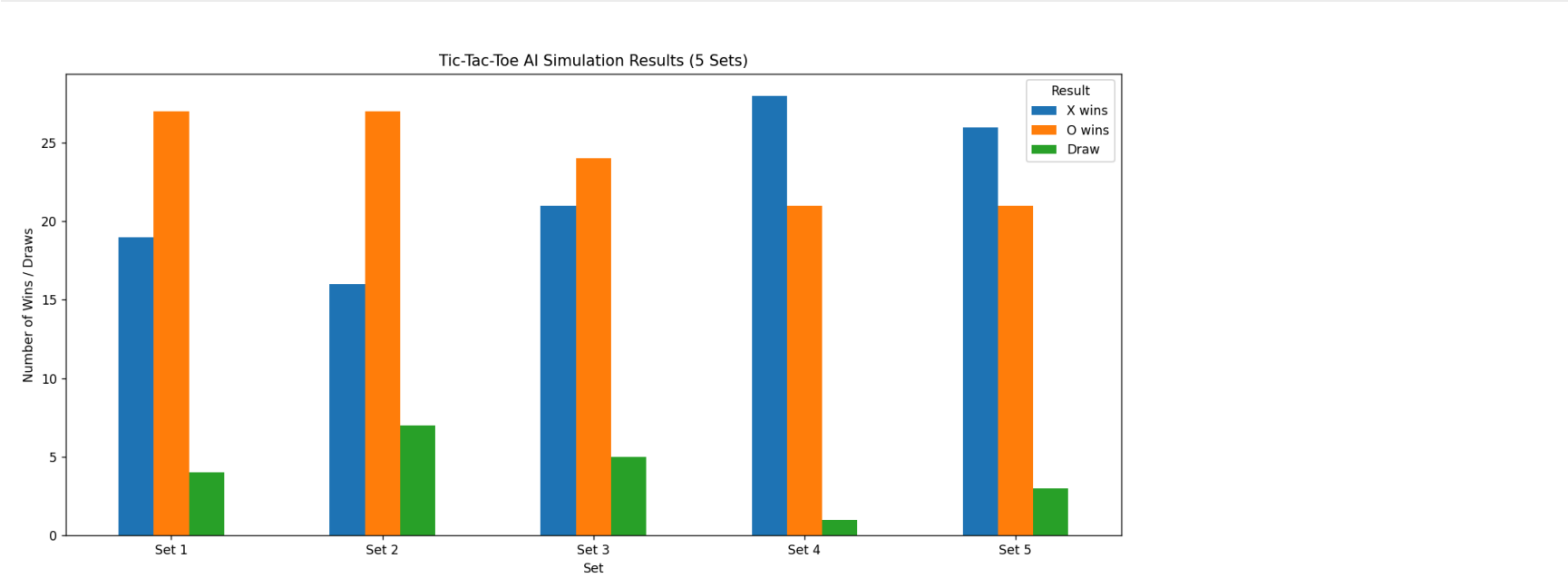
```
Set 1: {'X wins': 19, 'O wins': 27, 'Draw': 4}
Set 2: {'X wins': 16, 'O wins': 27, 'Draw': 7}
Set 3: {'X wins': 21, 'O wins': 24, 'Draw': 5}
Set 4: {'X wins': 28, 'O wins': 21, 'Draw': 1}
Set 5: {'X wins': 26, 'O wins': 21, 'Draw': 3}

=== Summary ===
      X wins  O wins  Draw
Set 1      19     27     4
Set 2      16     27     7
Set 3      21     24     5
Set 4      28     21     1
Set 5      26     21     3
PS D:\Github\tic-tac-toe-ai-battle>
```

[Image 1] Summary output of wins, losses, and draws for each set

[Table 1] Summary of O/X wins and draws for each set

Set	X wins	O wins	Draws
1	19	27	4
2	16	27	7
3	21	24	5
4	28	21	1
5	26	21	3



[Image 2] Bar graph visualizing the results of 5 sets using matplotlib

In Contrast, when randomness was removed and both AIs played purely optimally, all 50 games ended in a draw.

```
Simulated 50 games:
{'X wins': 0, 'O wins': 0, 'Draw': 50}
PS D:\Github\tic-tac-toe-ai-battle>
```

[Image 3] Results when randomness was removed (all games ended in a draw)

4. Conclusion

As shown in the previous results, I introduced randomness into the simple structure of a Tic-Tac-Toe game to intentionally create an imperfect environment and observe how the AI would attempt to choose optimal moves.

When designing this project, I wanted to explore what would happen when intentional imperfection is introduced into a scenario where perfect AIs—like an unstoppable force and an immovable object—face off against each other.

The results showed that the introduction of randomness led to a meaningful difference in the distribution of wins and losses.

Through this experience, I realized that even in a simple structure, adding a small variable and changing the perspective can lead to completely different outcomes.

This insight suggests that in AI and security environments as well, unpredictable factors can always be present and must be taken into account.

Additional Experience : *webshell-upload-vuln Project*

After completing the team project, I independently launched an additional project where I built a separate backup environment to explore potential threat scenarios by analyzing how vulnerabilities could be chained together.

 **More details can be found at :**

<https://github.com/BarnOwl5/webshell-upload-vuln>

Although this was a relatively small-scale experiment, it sparked a meaningful shift in my perspective. I aim to continue fostering a flexible approach to security challenges moving forward.

Thank you for reading!